



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

ECA1423 – EMBEDDED SYSTEMS

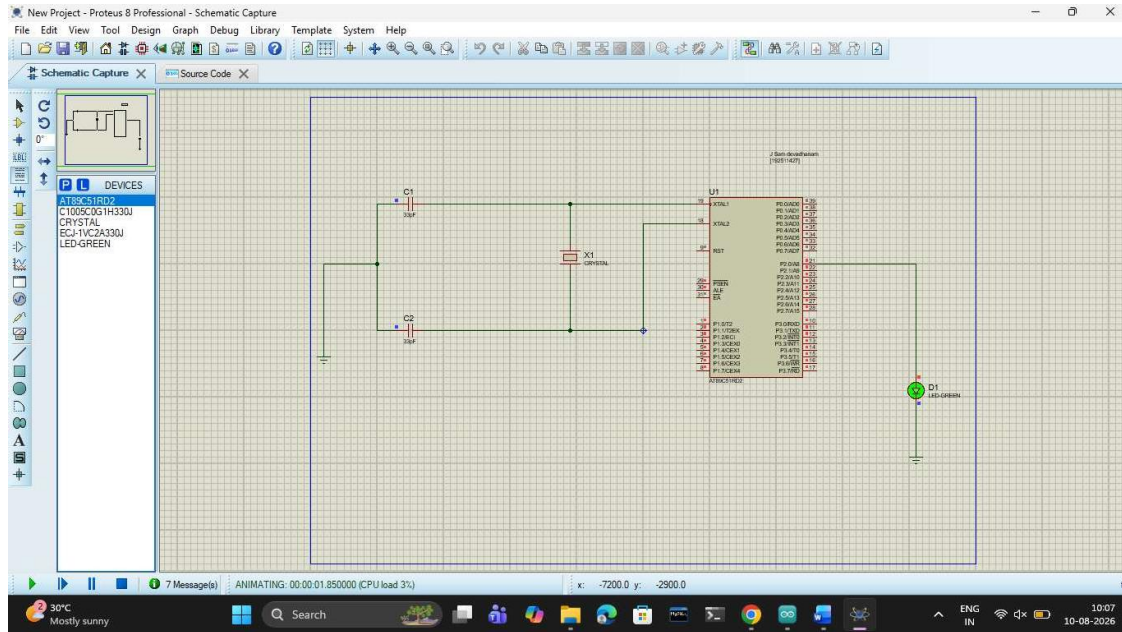
NAME : SURENDHAR G

REG. No: 192513079

LAB EXPERIMENT OUTPUT

EXP No: 1

FLASHING OF LED USING AT89C51 MICROCONTROLLER USING PROTEUS



C:\Keil_v5\C51\Examples\HELLO\LED1.uvproj - µVision

File Edit View Project Flash Debug Peripherals Tools SVCS Window

Target 1

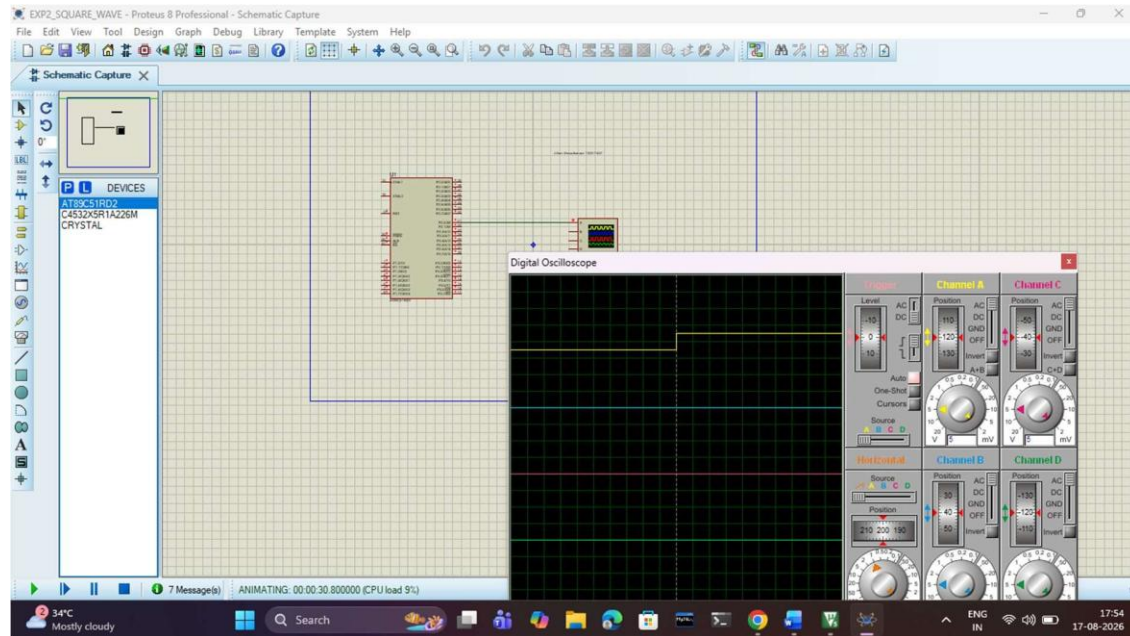
Project: LED1

- Target 1
 - Source Group 1
 - LED1.asm

```
1 ORG 0000H
2 UP: SETB P2.0
3 ACALL DELAY
4 CLR P2.0
5 ACALL DELAY
6 SJMP UP
7 DELAY: MOV R4, #35
8 H1: MOV R3, #255
9 H2: DJNZ R3, H2
10 DJNZ R4, H1
11 RET
12 END
```

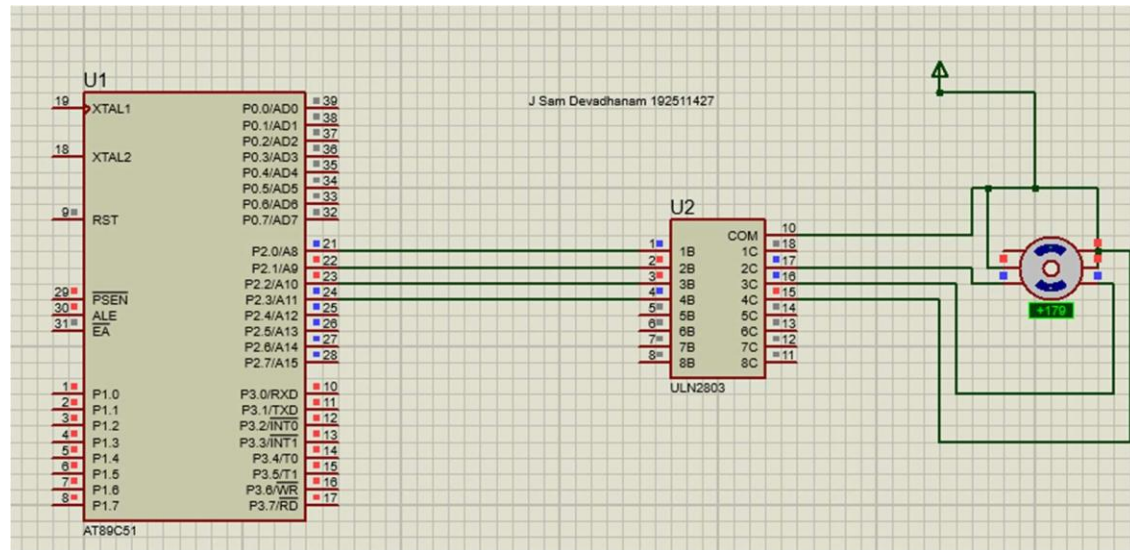
EXP No: 2

GENERATION OF SQUARE WAVE USING PROTEUS



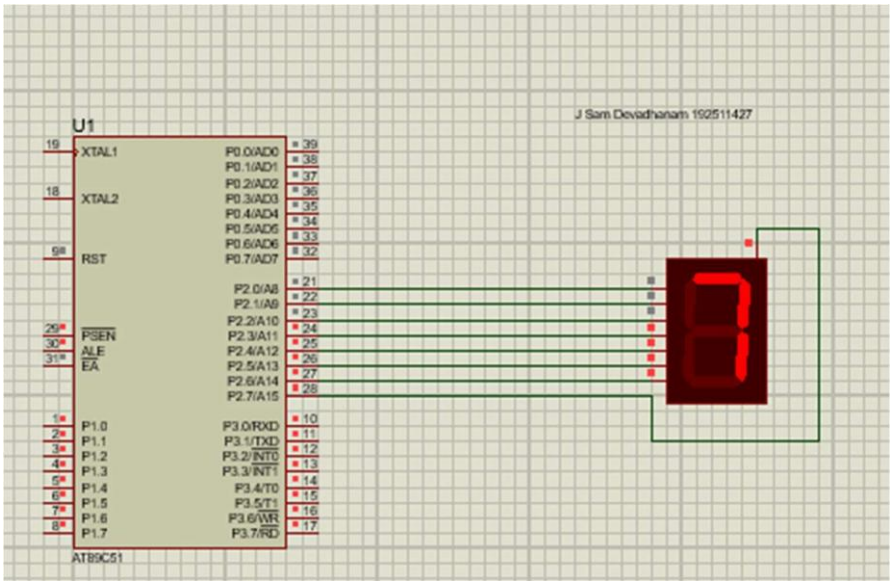
EXP No: 3

STEPPER MOTOR USING AT89C51 USING PROTEUS



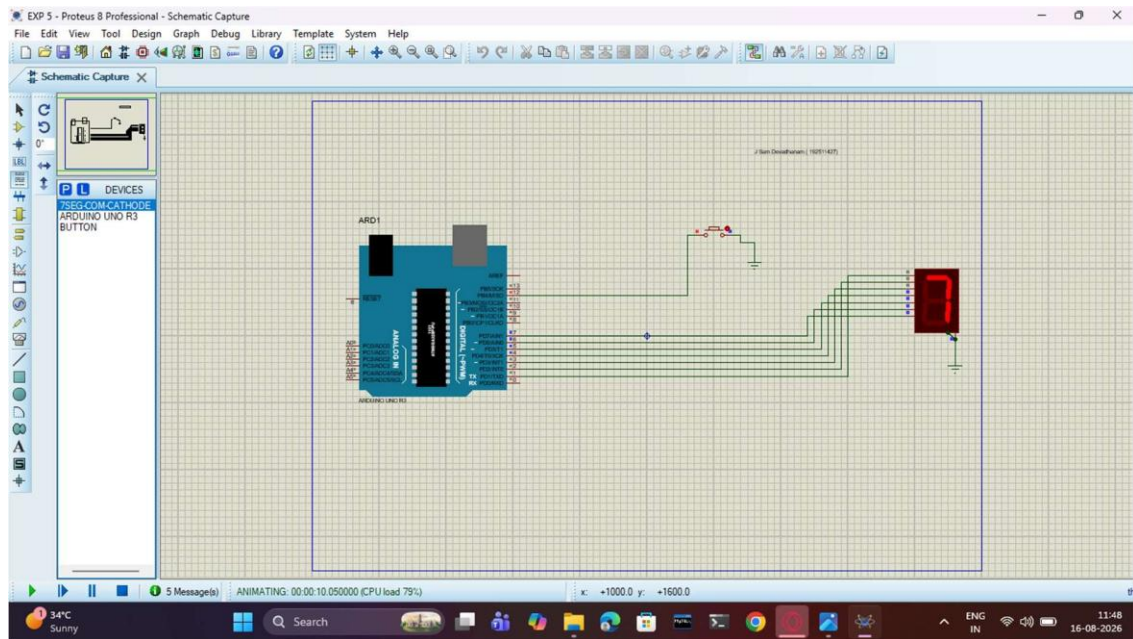
EXP No: 4

7 SEGMENT DISPLAY USING AT89C51 USING PROTEUS



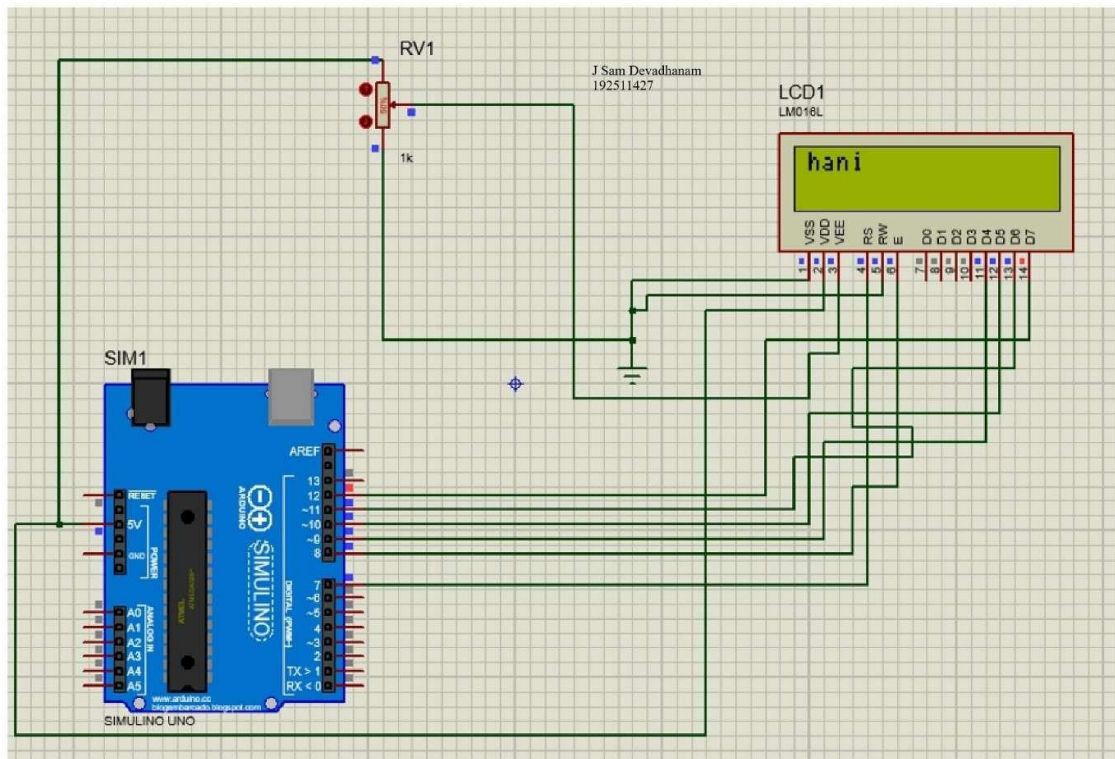
EXP No: 5

ARDUINO BASED MANUAL ELECTRONIC COUNTER USING PROTEUS



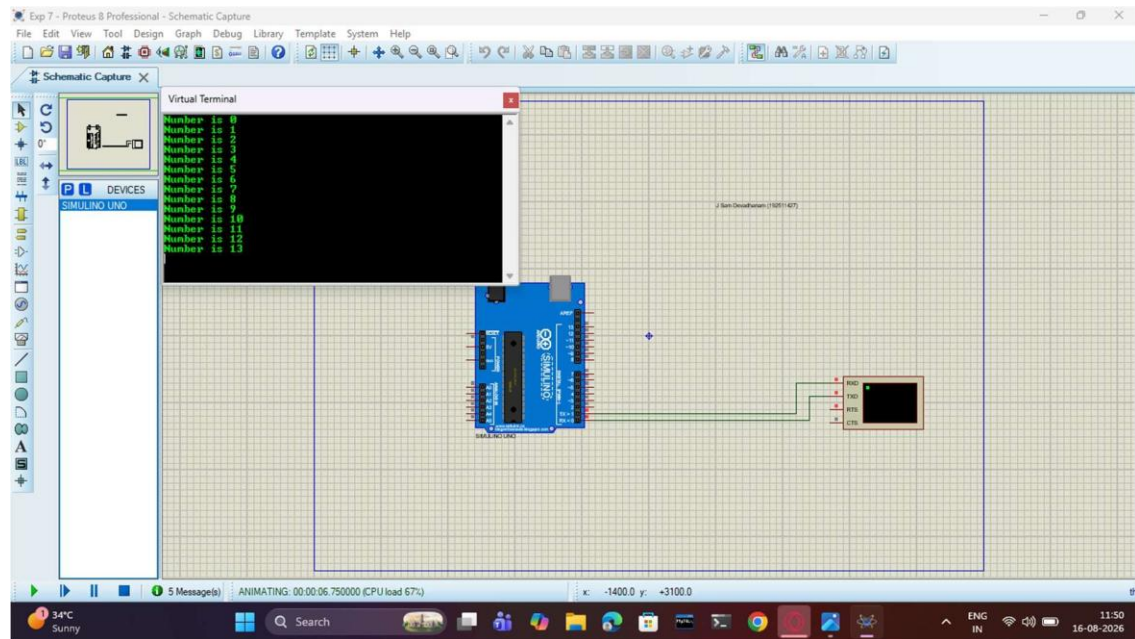
EXP No: 6

ARDUINO TO 16x2 LCD DISPLAY USING PROTEUS



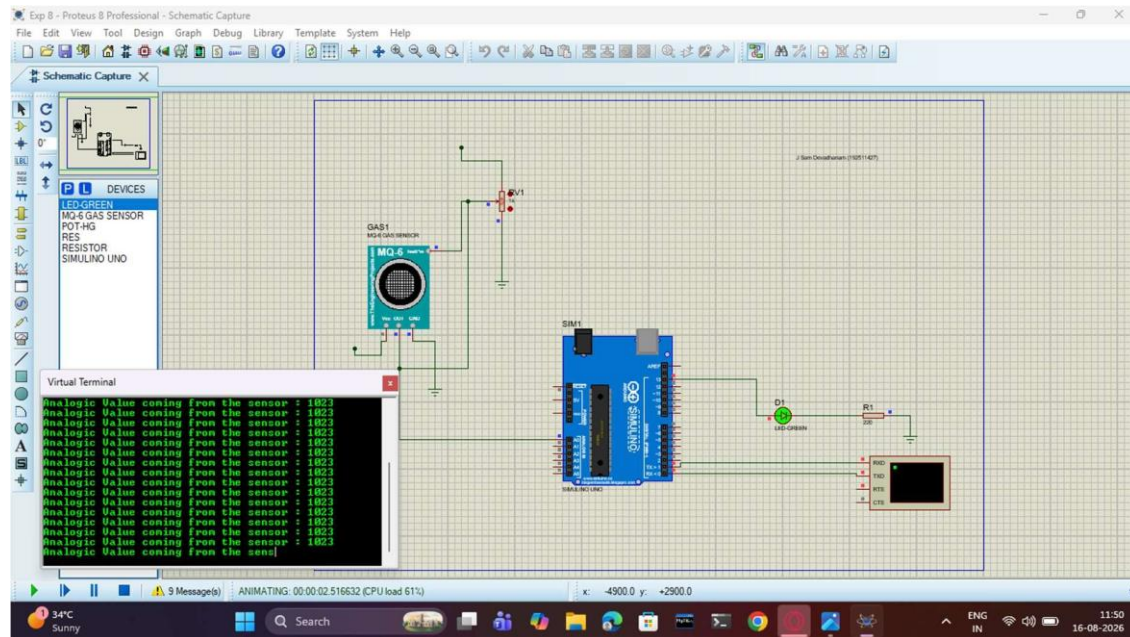
EXP No: 7

SERIAL COMMUNICATION USING ARDUINO WITH PROTEUS



EXP No: 8

GAS SENSOR MQ-2 IN PROTEUS USING ARDUINO

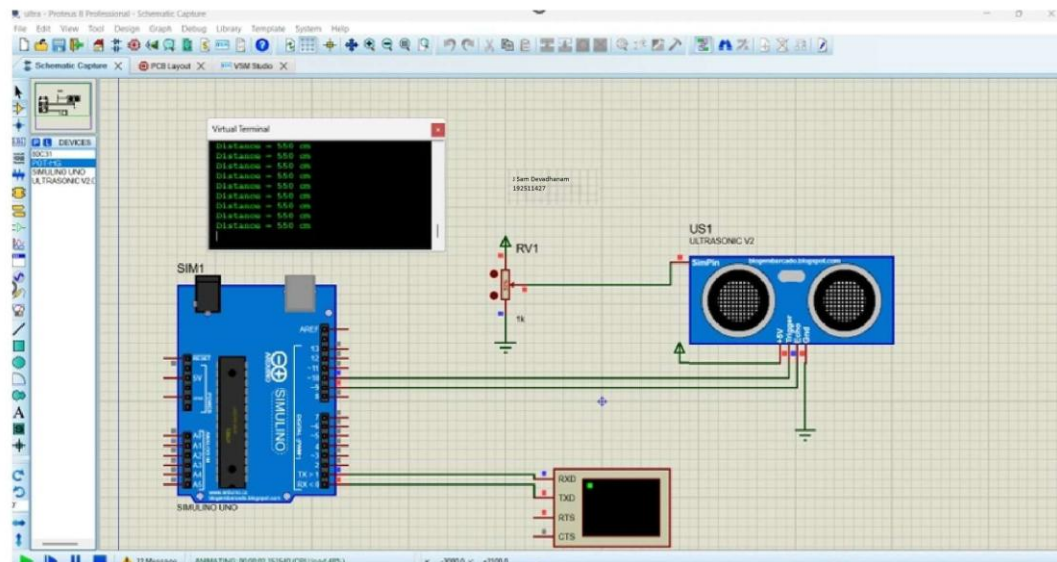


EXP No: 9

ULTRASONIC SENSOR IN PROTEUS USING ARDUINO

AIM:

To write an Embedded C program for interfacing Ultrasonic Sensor using Arduino Uno and Proteus

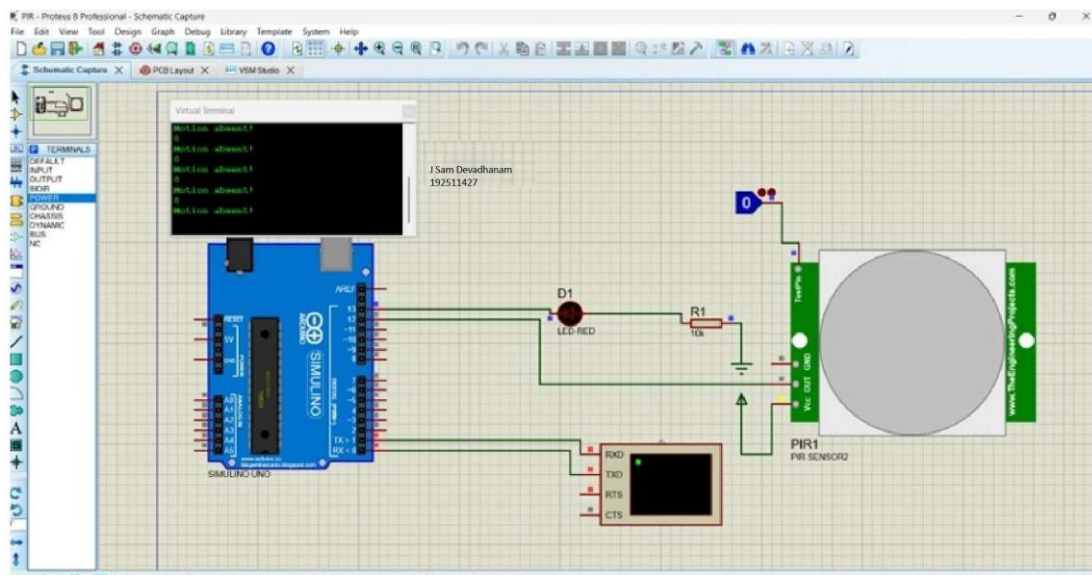


Exp No: 10

PIR SENSOR IN PROTEUS USING ARDUINO

AIM:

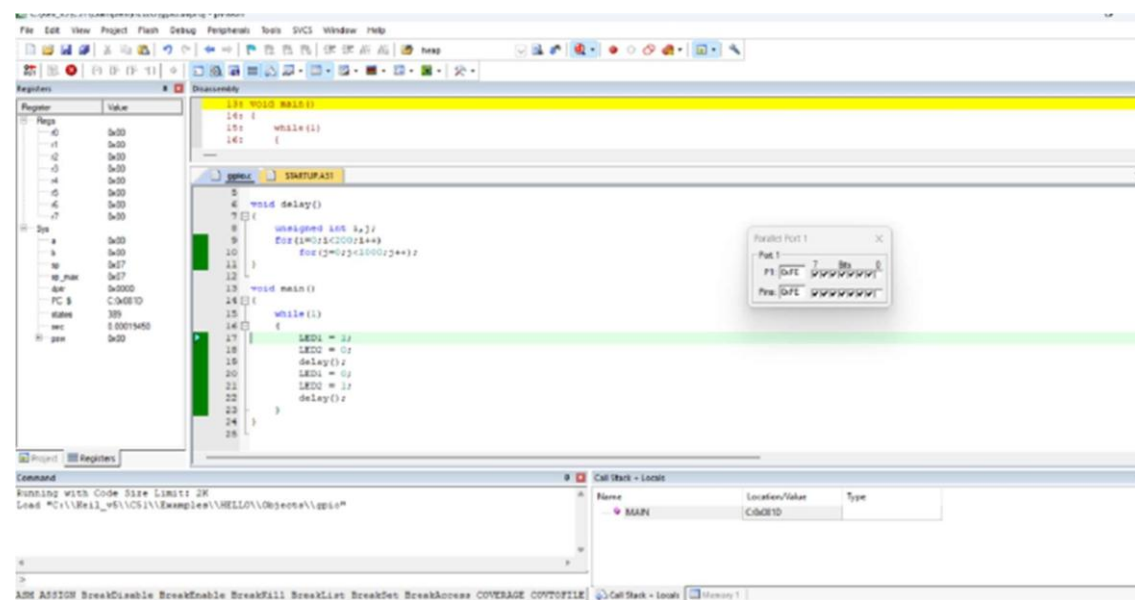
To write an Embedded C program for interfacing PIR Sensor using Arduino Uno and Proteus.



STUDY OF GPIO & BIT ADDRESSING IN AT89C51

AIM:

To study GPIO control and bit-addressable SFR using Embedded C.



EXP No: 12

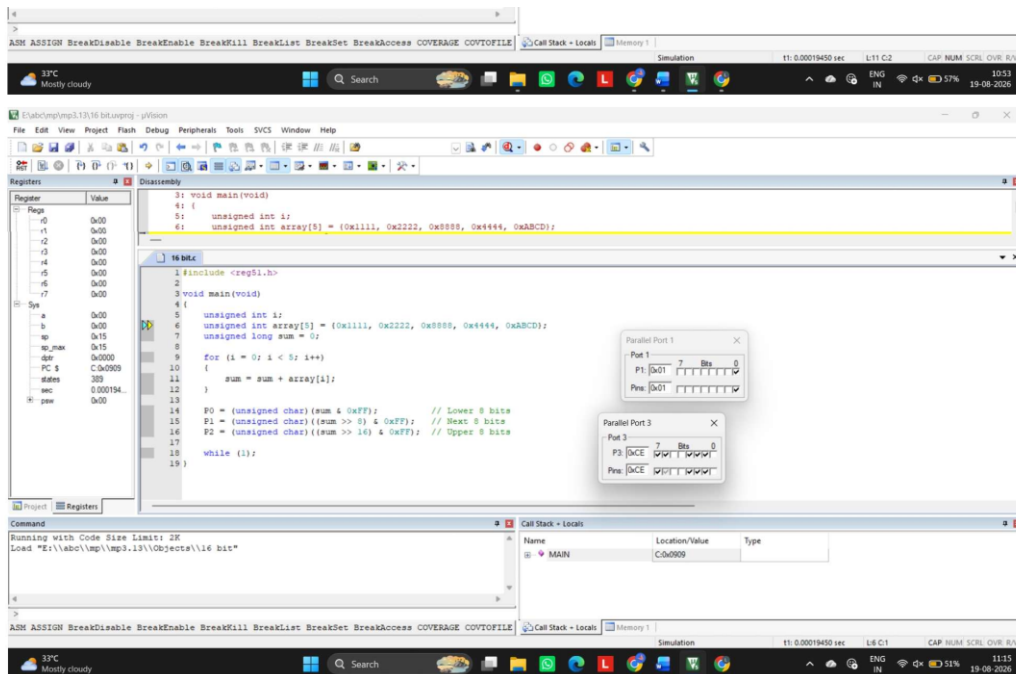
EMBEDDED C PROGRAM TO MULTIPLY TWO 16-BIT BINARY NUMBERS

AIM:

TO WRITE AN EMBEDDED C PROGRAM TO MULTIPLY TWO 16-BIT NUMBERS USING KEIL SOFTWARE

SOFTWARE REQUIRED:

- KEIL



EXP No: 13

EMBEDDED C PROGRAM TO ADD AN ARRAY OF 16-BIT NUMBERS AND

STORE THE 32-BIT RESULT IN INTERNAL RAM

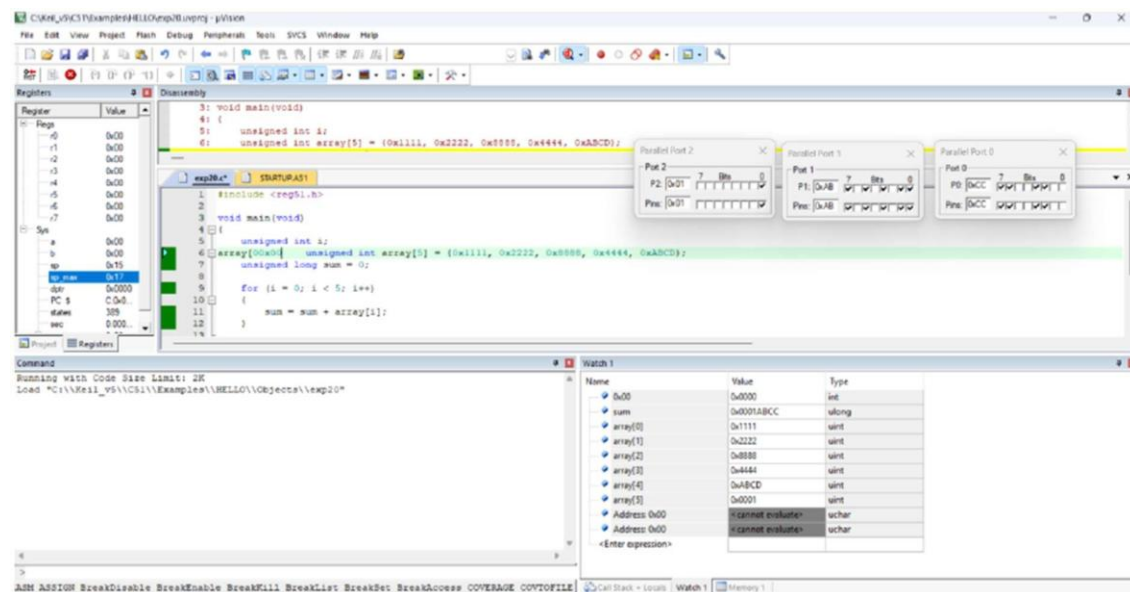
AIM:

TO WRITE AN EMBEDDED C PROGRAM TO ADD AN ARRAY OF 16-BIT NUMBERS AND STORE THE RESULTS IN

INTERNAL RAM USING KEIL SOFTWARE.

SOFTWARE REQUIRED:

- KEIL



EXP No: 14

EMBEDDED C PROGRAM TO DISPLAY “HELLO WORLD” MESSAGE

AIM:

TO WRITE AN EMBEDDED C PROGRAM TO DISPLAY “HELLO WORLD” MESSAGE USING KEIL SOFTWARE.

SOFTWARE REQUIRED:

- KEIL SOFTWARE

The screenshot displays a disassembler window with the following assembly code:

```

2: void main (void){
3: SCON = 0x50; /* SCON: mode 1, 8-bit UART, enable rcvr */ TMOD = 0x20;
C:0x0C11 759850 MOV     SCON(0x98),#0x50
C:0x0C14 759920 MOV     TMOD(0x89),#0x20
...
1#include <reg51.h> #include <stdio.h>
2void main (void){
3SCON = 0x50; /* SCON: mode 1, 8-bit UART, enable rcvr */ TMOD = 0x20;
4/* TMOD: timer 1, mode 2, 8-bit reload */ TH1 = 0xFD; /* TH1: reload
5
6value for 9600 baudrate */
7TH1 = 1; /* TH1: timer 1 run */
8TI = 1; /* TI: set TI to send first char of UART */ while (1)
9{
10printf ("Hello World ! \n ");
11}
12}

```

Below the code, the UART1 pin is shown connected to a 9-pin D-sub connector. The pin is labeled "UART #1".

Exp No: 15

EMBEDDED C PROGRAM TO CONVERT THE HEXADECIMAL DATA 0XCFH TO DECIMAL AND DISPLAY THE DIGITS ON PORTS P0, P1 AND P2

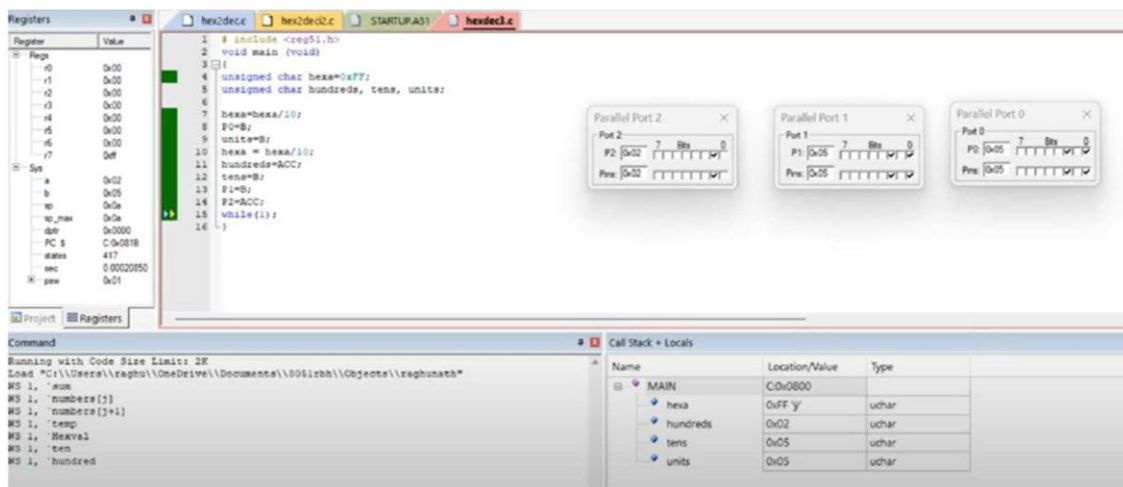
AIM:

TO WRITE AN EMBEDDED C PROGRAM TO CONVERT HEXADECIMAL NUMBERS INTO DECIMAL AND

DISPLAY THE DIGITS ON PORTS USING KEIL SOFTWARE.

SOFTWARE REQUIRED:

- KEIL SOFTWARE



Exp No: 16

SERIAL INTERRUPT USING UART (KEYBOARD INPUT) IN KEIL

AIM:

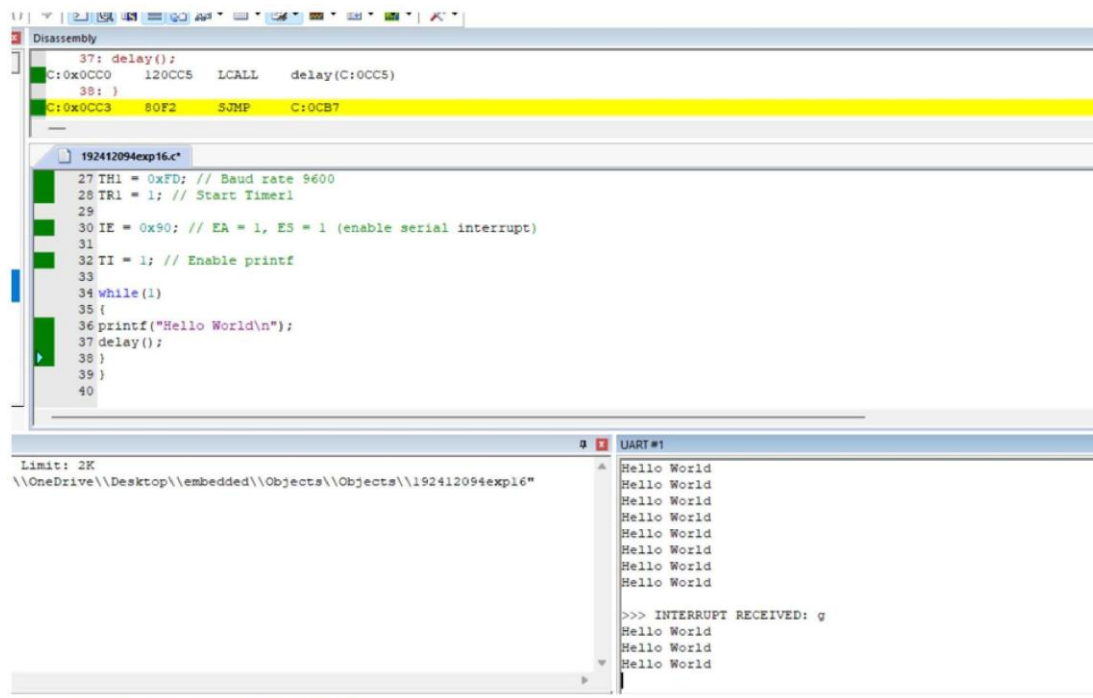
TO WRITE AND SIMULATE AN EMBEDDED C PROGRAM USING SERIAL INTERRUPT IN 8051

MICROCONTROLLER SUCH THAT WHILE A MESSAGE IS CONTINUOUSLY TRANSMITTED, ANY KEY PRESSED FROM

THE KEYBOARD GENERATES AN INTERRUPT AND IS DISPLAYED THROUGH UART.

APPARATUS / SOFTWARE REQUIRED:

- PC WITH WINDOWS OS
- KEIL MVISION IDE
- 8051 SIMULATOR (KEIL BUILT-IN)
- SERIAL WINDOW (UART #1)



The screenshot displays the Keil MVision IDE interface. The top pane shows the disassembly of the program, with the following instructions highlighted:

```
37: delay();  
C:0x0CC0 120CC5 LCALL delay(C:0CC5)  
38: }  
C:0x0CC3 80F2 SJMP C:0CB7
```

The bottom pane shows the source code of the program, which is a C program for an 8051 microcontroller. The code includes comments and a while loop that prints "Hello World" and calls a delay function. The code is as follows:

```
192412094exp16.c  
27 TH1 = 0xFD; // Baud rate 9600  
28 TR1 = 1; // Start Timer1  
29  
30 IE = 0x90; // EA = 1, ES = 1 (enable serial interrupt)  
31  
32 TI = 1; // Enable printf  
33  
34 while(1)  
35 {  
36 printf("Hello World\n");  
37 delay();  
38 }  
39 }  
40
```

The bottom right pane shows the UART #1 window, which displays the output of the program. The output consists of multiple "Hello World" messages, followed by a prompt "INTERRUPT RECEIVED: g".

Exp No: 17

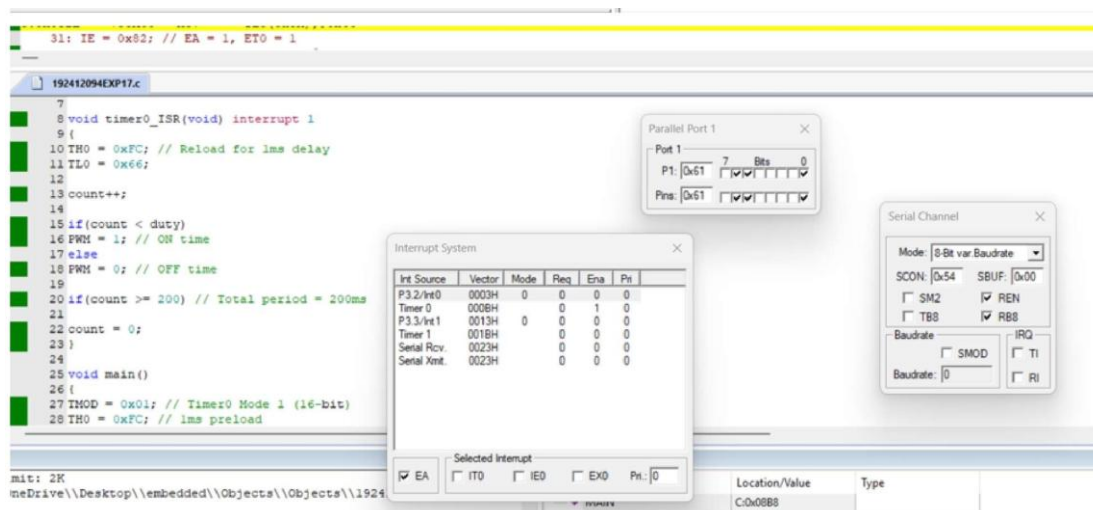
UART INTERFACING USING INTERRUPT IN KEIL

AIM:

TO RECEIVE SERIAL DATA USING INTERRUPT.

APPARATUS REQUIRED:

- PC WITH WINDOWS OS
- KEIL MVISION IDE
- 8051 SIMULATOR (KEIL BUILT-IN)
- SERIAL WINDOW (UART #1)



EXP No: 18

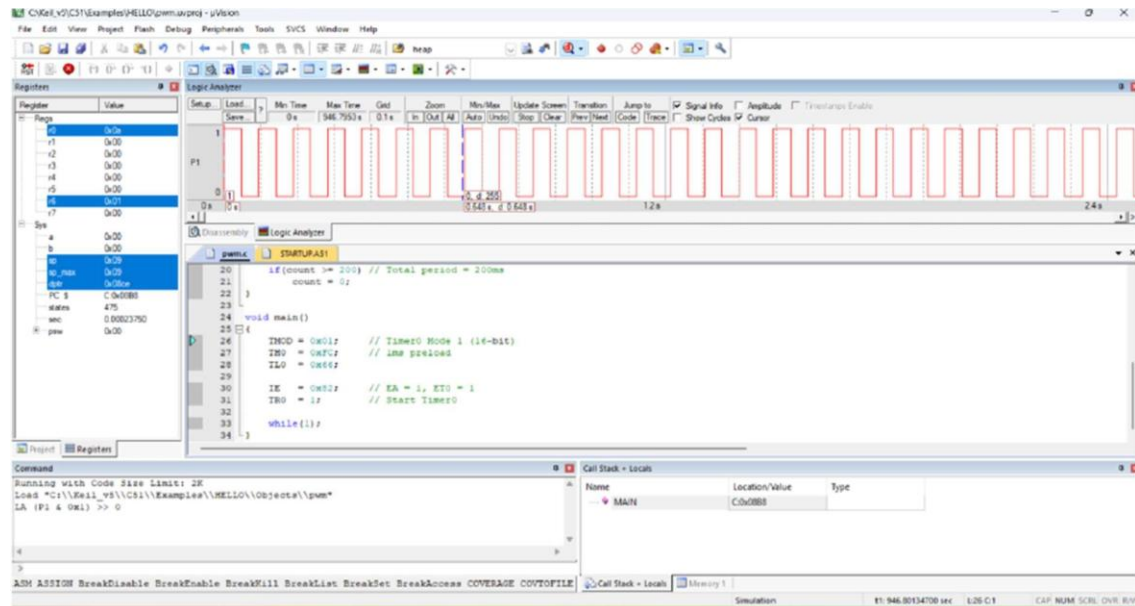
GENERATION OF PWM USING TIMER INTERRUPT

AIM:

TO GENERATE PWM USING TIMER INTERRUPT.

APPARATUS / SOFTWARE REQUIRED:

- PC WITH WINDOWS OS
- KEIL MVISION IDE
- 8051 SIMULATOR (KEIL BUILT-IN)
- SERIAL WINDOW (UART #1)



Exp No: 19

BLINKING LEDS USING SOFTWARE DELAY ROUTINE IN LPC2148 KIT

AIM:

TO WRITE AND EXECUTE A C PROGRAM TO BLINK LEDS USING SOFTWARE DELAY ROUTINE IN LPC

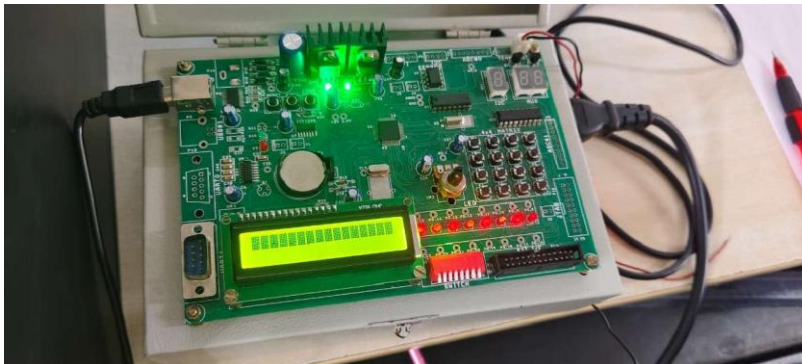
2148 KIT

APPARATUS REQUIRED:

KEIL UVISION5 SOFTWARE

PHILIPS FLAH PROGRAMMER

LPC 2148 KIT



Exp No: 20

**PROGRAM TO READ THE SWITCH AND DISPLAY IN THE LEDS
USING LPC2148 KIT**

AIM:

TO WRITE AND EXECUTE C PROGRAM TO READ THE SWITCH AND DISPLAY IN THE
LEDS USING

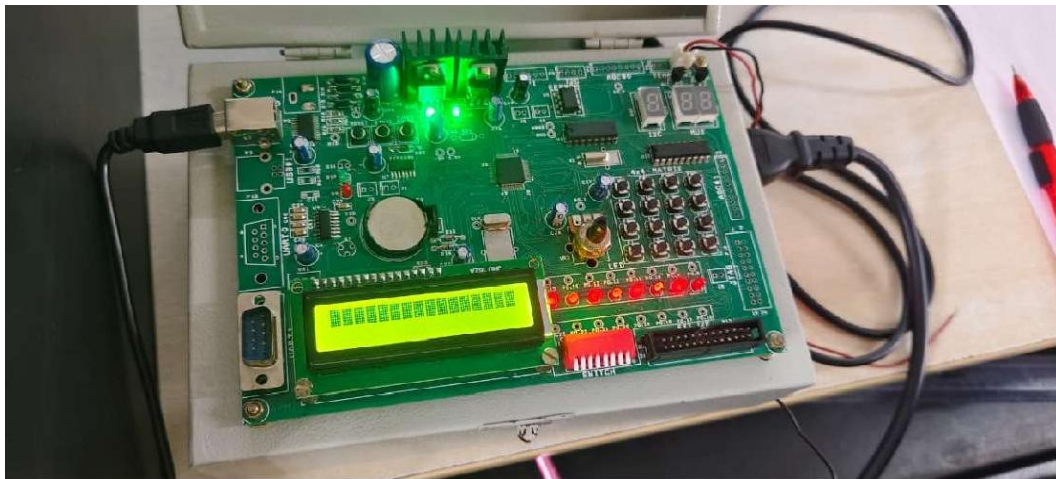
LPC2148 KIT

APPARATUS REQUIRED:

KEIL uVISION5 SOFTWARE

PHILIPS FLAH PROGRAMMER

LPC 2148 KIT



Exp No: 21

DISPLAY A NUMBER IN SEVEN SEGMENT LED IN LPC2148 KIT

AIM:

TO WRITE AND EXECUTE C PROGRAM TO DISPLAY A NUMBER IN SEVEN SEGMENT LED IN

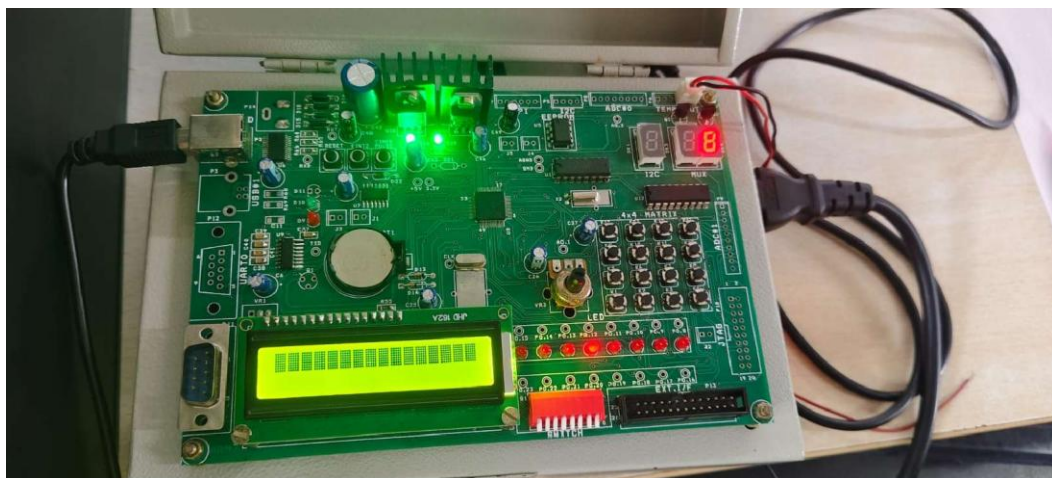
LPC2148 KIT

APPARATUS REQUIRED:

KEIL uVISION5 SOFTWARE

PHILIPS FLAH PROGRAMMER

LPC 2148 KIT



EXP No: 22

SERIAL TRANSMISSION AND RECEPTION USING ON-CHIP UART IN LPC2148 KIT

AIM:

TO WRITE AND EXECUTE C PROGRAM FOR SERIAL TRANSMISSION AND RECEPTION USING ON-CHIP

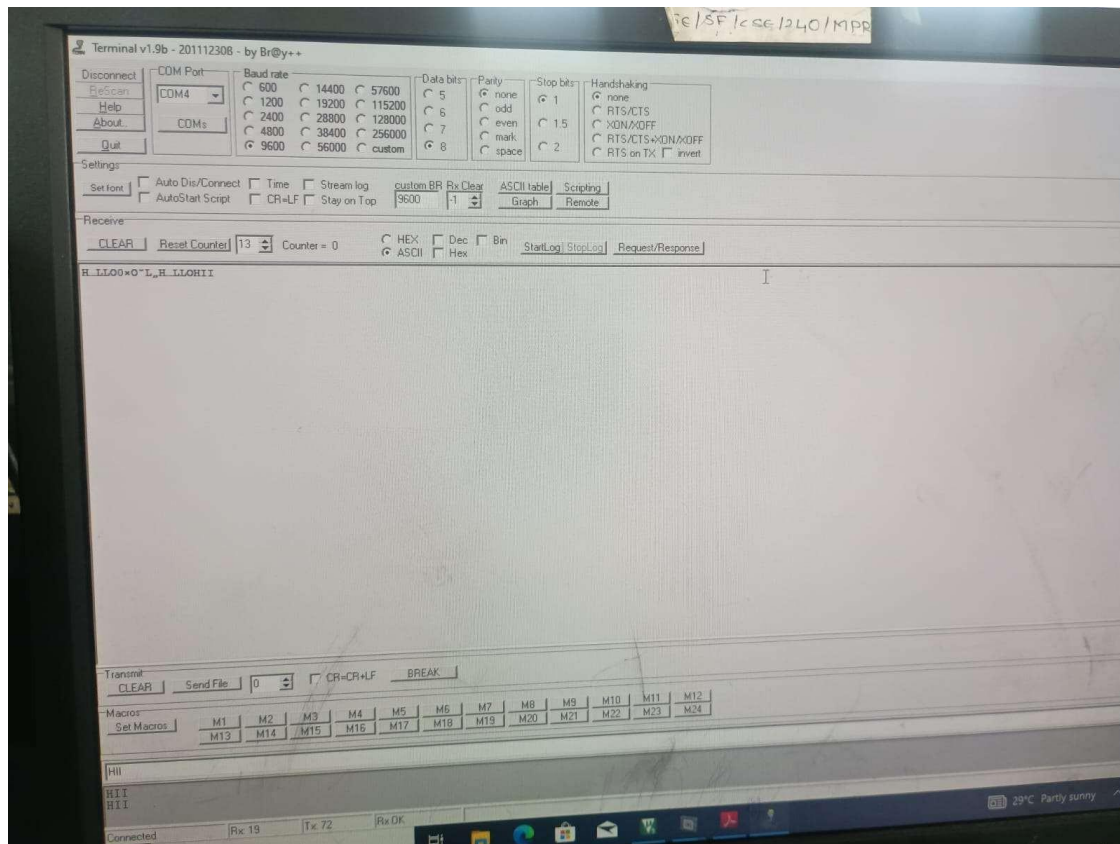
UART IN LPC2148 KIT.

APPARATUS REQUIRED:

KEIL uVISION5 SOFTWARE

PHILIPS FLAH PROGRAMMER

LPC 2148 KIT



EXP No: 23

ACCESS AN INTERNAL ADC AND DISPLAY THE BINARY OUTPUT IN LEDS IN LPC2148 KIT

AIM:

TO WRITE AND EXECUTE C PROGRAM FOR ACCESSING AN INTERNAL ADC AND
DISPLAY THE BINARY

OUTPUT IN LEDS IN LPC2148 KIT.

APPARATUS REQUIRED:

KEIL uVISION5 SOFTWARE

PHILIPS FLAH PROGRAMMER

LPC 2148 KIT



EXP No: 24

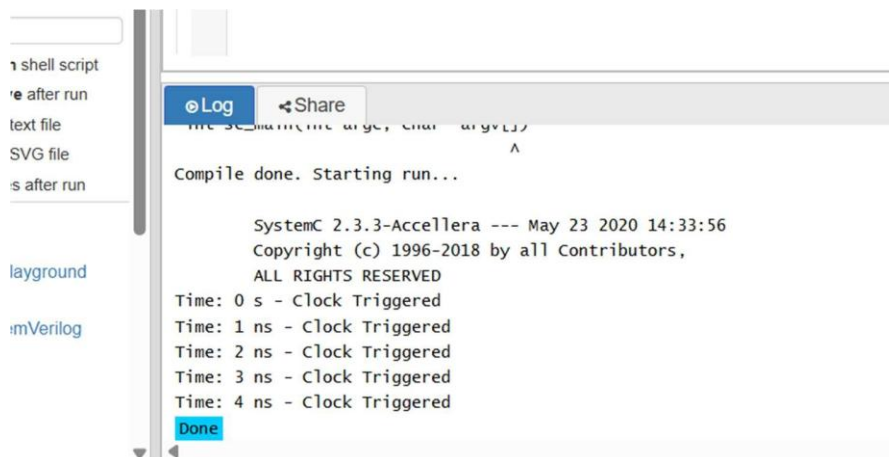
MODELING CLOCK-DRIVEN CONCURRENT PROCESSES USING SC_THREAD AND WAIT() IN SYSTEMC

AIM:

TO UNDERSTAND AND IMPLEMENT A CLOCK-DRIVEN PROCESS USING:

- SC_CLOCK
- SC_THREAD
- WAIT()
- EVENT SENSITIVITY (POSITIVE EDGE TRIGGER)

IN SYSTEMC.



EXP No: 25

PRODUCER-CONSUMER PROBLEM USING SYSTEMC

AIM:

TO IMPLEMENT AND SIMULATE THE PRODUCER-CONSUMER PROBLEM USING SYSTEMC.

SOFTWARE REQUIRED:

UBUNTU / WSL LINUX

G++ COMPILER

CMAKE

SYSTEMC LIBRARY



```
1 #include <systemc.h>
2 #include "design.cpp"
3
4 int sc_main(int argc, char* argv[])
5 {
6     // Create FIFO with capacity 5
7     sc_fifo<int> fifo(5);
8
9     // Create Producer and Consumer
10    Producer producer("Producer");
11    Consumer consumer("Consumer");
12
13    // Connect FIFO
14    producer.out(fifo);
15    consumer.in(fifo);
16
17    // Start simulation for 10 seconds
18    sc_start(10, SC_SEC);
19
20    return 0;
21 }
```

Log Share

Copyright (c) 1996-2020 by All Contributors,
ALL RIGHTS RESERVED

Produced: 1
Consumed: 1
Produced: 2
Consumed: 2
Produced: 3
Consumed: 3
Produced: 4
Consumed: 4
Produced: 5
Consumed: 5

Done

EXP No: 26

TRANSACTION LEVEL MODELING (TLM) USING SYSTEMC

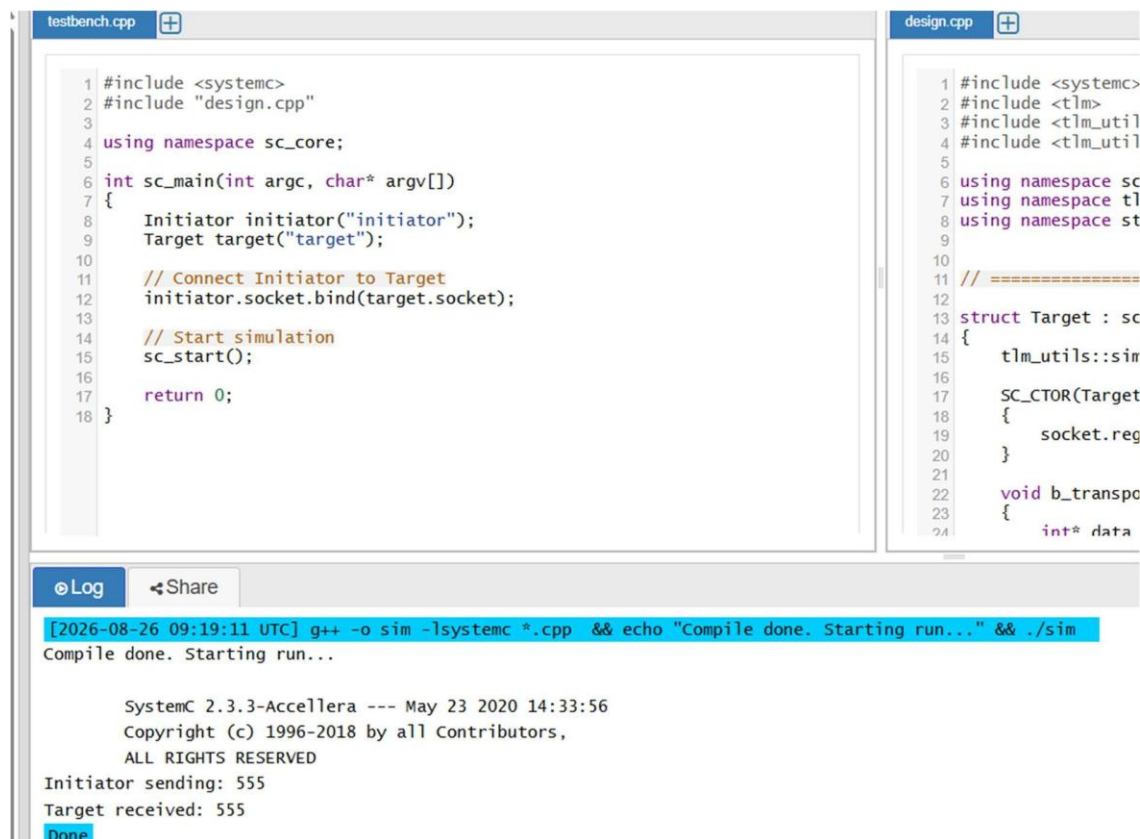
AIM:

TO MODEL AND SIMULATE A SIMPLE TRANSACTION-LEVEL MODEL (TLM) USING INITIATOR AND

TARGET MODULES IN SYSTEMC.

SOFTWARE REQUIRED:

- UBUNTU (OR WSL UBUNTU)
- G++
- CMAKE
- SYSTEMC WITH TLM-2.0 LIBRARY



```
testbench.cpp
1 #include <systemc>
2 #include "design.cpp"
3
4 using namespace sc_core;
5
6 int sc_main(int argc, char* argv[])
7 {
8     Initiator initiator("initiator");
9     Target target("target");
10
11     // Connect Initiator to Target
12     initiator.socket.bind(target.socket);
13
14     // Start simulation
15     sc_start();
16
17     return 0;
18 }
```

```
design.cpp
1 #include <systemc>
2 #include <tlm>
3 #include <tlm_util>
4 #include <tlm_util>
5
6 using namespace sc
7 using namespace tlm
8 using namespace st
9
10 // =====
11
12 struct Target : sc
13 {
14     tlm_utils::sint
15     SC_CTOR(Target
16     {
17         socket.reg
18     }
19
20 void b_transpo
21 {
22     int* data
```

Log Share

[2026-08-26 09:19:11 UTC] g++ -o sim -lsystemc *.cpp && echo "Compile done. Starting run..." && ./sim

Compile done. Starting run...

SystemC 2.3.3-Accellera --- May 23 2020 14:33:56

Copyright (c) 1996-2018 by all Contributors,

ALL RIGHTS RESERVED

Initiator sending: 555

Target received: 555

Done

EXP No: 27

BINARY SEMAPHORE IN SYSTEMC

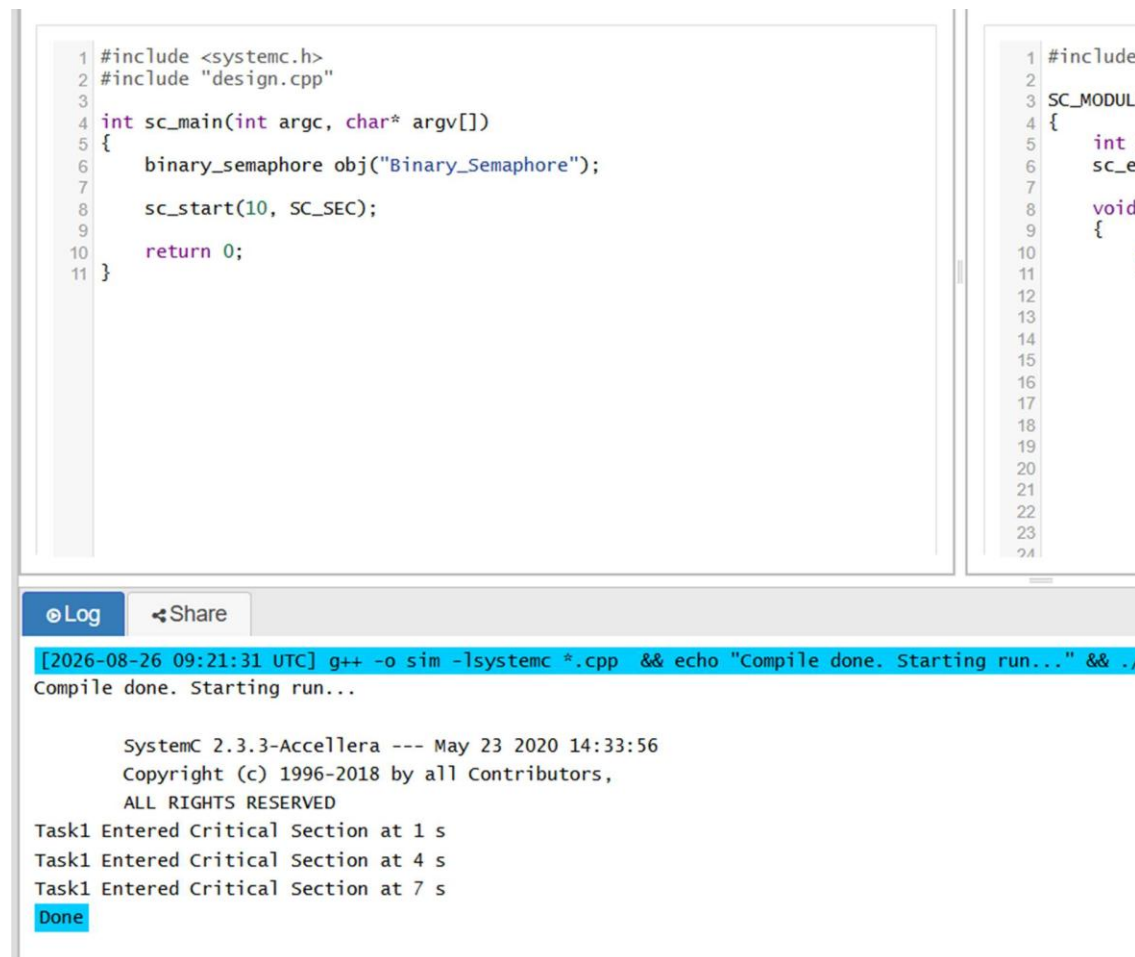
AIM:

TO IMPLEMENT AND SIMULATE A BINARY SEMAPHORE USING SYSTEMC FOR TASK

SYNCHRONIZATION.

SOFTWARE REQUIRED:

- UBUNTU (OR WSL UBUNTU)
- G++
- CMAKE
- SYSTEMC



The screenshot displays a SystemC simulation environment with two source files and their execution output.

Left File (design.cpp):

```
1 #include <systemc.h>
2 #include "design.cpp"
3
4 int sc_main(int argc, char* argv[])
5 {
6     binary_semaphore obj("Binary_Semaphore");
7
8     sc_start(10, SC_SEC);
9
10    return 0;
11 }
```

Right File (SC_MODULE.h):

```
1 #include
2
3 SC_MODULE
4 {
5     int :
6     sc_e
7
8     void
9     {
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
```

Execution Output:

```
[2026-08-26 09:21:31 UTC] g++ -o sim -lsystemc *.cpp && echo "Compile done. Starting run..." && ./
Compile done. Starting run...

SystemC 2.3.3-Accellera --- May 23 2020 14:33:56
Copyright (c) 1996-2018 by all Contributors,
ALL RIGHTS RESERVED
Task1 Entered Critical Section at 1 s
Task1 Entered Critical Section at 4 s
Task1 Entered Critical Section at 7 s
Done
```

EXP No:28

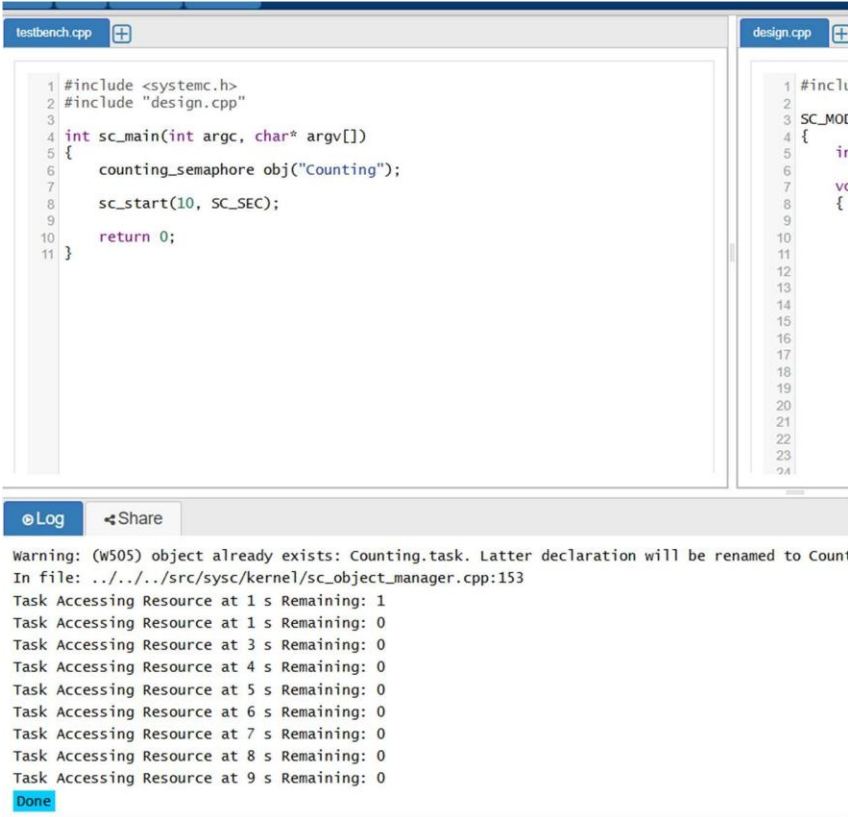
COUNTING SEMAPHORE IN SYSTEMC

AIM:

TO IMPLEMENT COUNTING SEMAPHORE ALLOWING MULTIPLE RESOURCE ACCESS.

SOFTWARE REQUIRED:

- UBUNTU (OR WSL UBUNTU)
- G++
- CMAKE
- SYSTEMC



```
testbench.cpp
1 #include <systemc.h>
2 #include "design.cpp"
3
4 int sc_main(int argc, char* argv[])
5 {
6     counting_semaphore obj("Counting");
7
8     sc_start(10, SC_SEC);
9
10    return 0;
11 }

design.cpp
1 #inclu
2
3 SC_MOD
4 {
5     in
6
7     vc
8     {
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
```

Warning: (W505) object already exists: Counting.task. Latter declaration will be renamed to Count
In file: ../../src/sysc/kernel/sc_object_manager.cpp:153

Task Accessing Resource at 1 s Remaining: 1
Task Accessing Resource at 1 s Remaining: 0
Task Accessing Resource at 3 s Remaining: 0
Task Accessing Resource at 4 s Remaining: 0
Task Accessing Resource at 5 s Remaining: 0
Task Accessing Resource at 6 s Remaining: 0
Task Accessing Resource at 7 s Remaining: 0
Task Accessing Resource at 8 s Remaining: 0
Task Accessing Resource at 9 s Remaining: 0

Done

EXP No: 29

PRIORITY-BASED INTERRUPT HANDLING

AIM:

TO MODEL PRIORITY-BASED INTERRUPT SERVICING.

SOFTWARE REQUIRED:

- UBUNTU (OR WSL UBUNTU)
- G++
- CMAKE
- SYSTEMC

```
1 #include <systemc.h>
2 #include "design.cpp"
3
4 int sc_main(int argc, char* argv[])
5 {
6     counting_semaphore obj("Counting");
7
8     sc_start(10, SC_SEC);
9
10    return 0;
11 }
```

```
1 #incl
2
3 SC_MO
4 {
5     i
6
7     v
8     {
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
```

Log Share

Copyright (c) 1996-2018 by all Contributors,
ALL RIGHTS RESERVED

Task Accessing Resource at 1 s Remaining: 1
Task Accessing Resource at 1 s Remaining: 0
Task Accessing Resource at 3 s Remaining: 0
Task Accessing Resource at 4 s Remaining: 0
Task Accessing Resource at 5 s Remaining: 0
Task Accessing Resource at 6 s Remaining: 0
Task Accessing Resource at 7 s Remaining: 0
Task Accessing Resource at 8 s Remaining: 0
Task Accessing Resource at 9 s Remaining: 0

Done

EXP No: 30

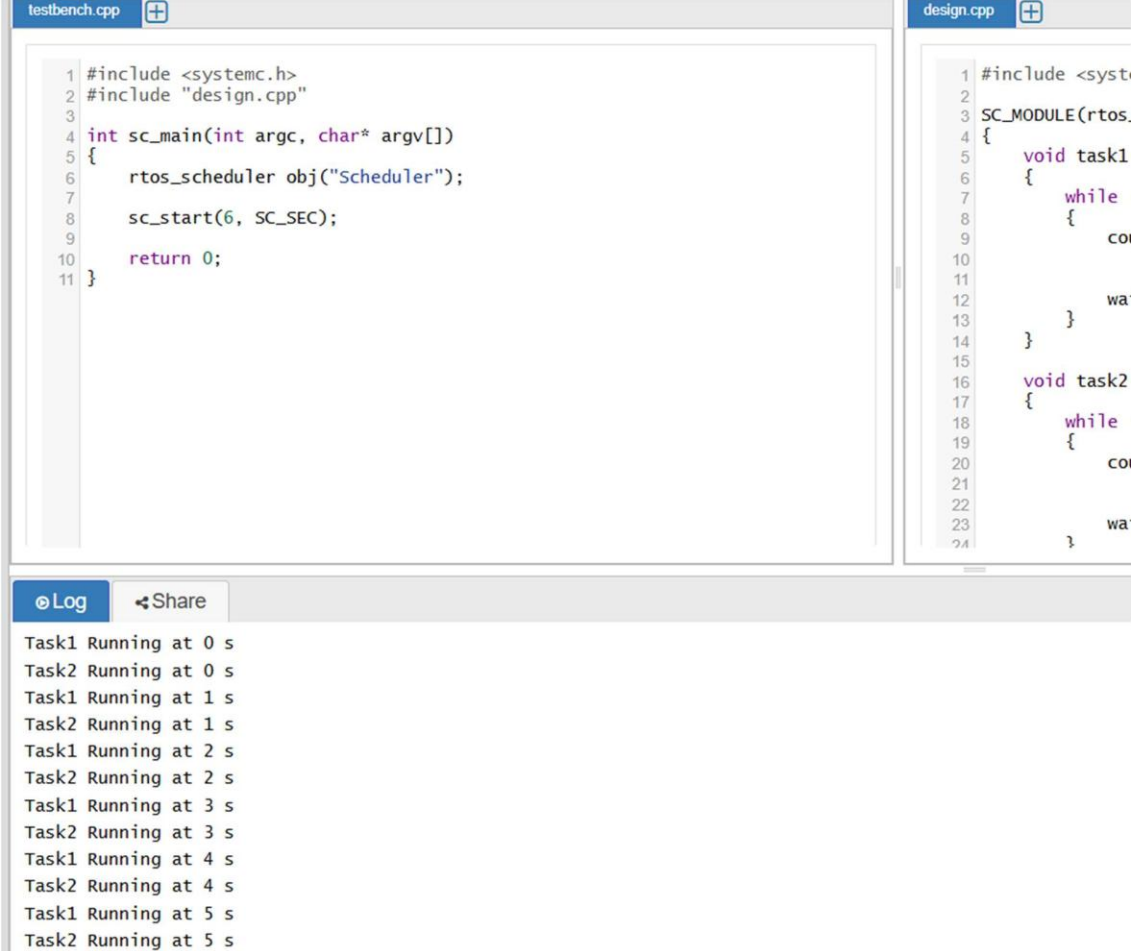
RTOS TASK SCHEDULING

AIM:

TO SIMULATE ROUND-ROBIN SCHEDULING.

SOFTWARE REQUIRED:

- UBUNTU (OR WSL UBUNTU)
- G++
- CMAKE
- SYSTEMC



```
testbench.cpp
1 #include <systemc.h>
2 #include "design.cpp"
3
4 int sc_main(int argc, char* argv[])
5 {
6     rtos_scheduler obj("Scheduler");
7
8     sc_start(6, SC_SEC);
9
10    return 0;
11 }
```

```
design.cpp
1 #include <systemc.h>
2
3 SC_MODULE(rtos_scheduler)
4 {
5     void task1()
6     {
7         while (true)
8         {
9             cout << "Task1 Running at " << sc_time_of_day() << endl;
10            wait(1, SC_SEC);
11        }
12    }
13
14    void task2()
15    {
16        while (true)
17        {
18            cout << "Task2 Running at " << sc_time_of_day() << endl;
19            wait(1, SC_SEC);
20        }
21    }
22 }
```

Log

Task1 Running at 0 s

Task2 Running at 0 s

Task1 Running at 1 s

Task2 Running at 1 s

Task1 Running at 2 s

Task2 Running at 2 s

Task1 Running at 3 s

Task2 Running at 3 s

Task1 Running at 4 s

Task2 Running at 4 s

Task1 Running at 5 s

Task2 Running at 5 s